AetherOps

AI-Powered Cloud Security & Governance Platform

Document	Progress Report — External Mentor Review Response
Project	AetherOps — B.Tech 3rd Year Project, CHARUSAT (DEPSTAR)
Team	Tushya Patel · Priyanshi Pujara · Janki Panchal · Dhruv Mistry
Reviewer	Mr. Amit Nayak (Mentor) , Ms. Mansi Solanki (External Mentor)
Date	September 2026

1. Current Build Status

The following table summarises the current completion state of each module. Work is active on all fronts — foundation components are done, core pipeline modules are in build, and the advanced features are designed and ready to implement.

Module	What Is Done	Status
Foundation & Setup	PostgreSQL schema, Docker environment, GitHub Actions CI pipeline, project repository structure	☑ Complete
AWS Connectivity	STS AssumeRole integration, read-only IAM role assumption, boto3 client initialised, connection verified against sandbox AWS account	☑ Complete
Resource Discovery	EC2, S3, RDS, IAM, Lambda discovery via boto3 implemented. Resources and relationships being written to PostgreSQL.	🔄 In Progress
Ingestion API	FastAPI endpoints for receiving and storing discovered resource data. Basic routes functional.	🔄 In Progress
User Authentication	JWT-based login, bcrypt password hashing, role assignment (Admin / Auditor / Viewer). Login flow working.	☑ Complete
Security Rules Engine	YAML rule schema defined. Rule loader implemented. First 4 rules written and tested against sample data.	🔄 In Progress
Attack-Path Correlation	Data model and graph traversal approach finalised. NetworkX integration planned for Week 4.	☐ Designed
Billing Integration	Cost Explorer API call structure written. Awaiting complete resource discovery before full integration.	☐ Designed
Multi-Cloud Pricing	AWS, Azure, GCP pricing APIs identified. SKU mapping table drafted. Implementation follows billing.	☐ Designed
AI Report Generation	Gemini API connected and tested. Prompt schema drafted. Full integration follows security module.	☐ Designed

Module	What Is Done	Status
AI Chat Bot	Architecture designed. Will be implemented after report generation is stable.	☐ Designed
React Dashboard	Component structure planned. Building alongside backend modules.	🔄 In Progress
Audit Trail	audit_log table created. log_action() function implemented. Wired to login and scan events.	☑ Complete

Legend: ☑ Complete — working and tested. 🔄 In Progress — actively being built this week. ☐ Designed — architecture finalised, implementation scheduled.

2. Response to External Mentor Feedback (Ms. Mansi Solanki)

Each point from the Review 1 feedback has been addressed below — with what we have already implemented, what we are changing in the current build, and where a decision was made to take a different approach with justification.

Feedback 1 — Leverage Native AWS Authentication & RBAC (AWS Cognito)

Recommendation: Replace custom user authentication with AWS Cognito and native IAM roles.

Steps taken based on this feedback:

- The AWS account connection already uses native STS AssumeRole — no customer credentials are stored by AetherOps. The customer creates a read-only IAM role and provides only the Role ARN. This is the native AWS cross-account access pattern and directly addresses the security concern.
- We have added explicit documentation of this flow in our architecture document so the read-only, least-privilege nature of the AWS connection is clearly visible.

Our approach for dashboard authentication:

- For the dashboard login, we have implemented JWT + bcrypt — and we have documented AWS Cognito as the recommended upgrade for a production deployment of AetherOps. Ms. Solanki's recommendation highlighted exactly why this upgrade matters at scale, and it is now part of our formal production roadmap.

Action taken: Architecture document updated with an explicit section on the IAM trust model — showing that AetherOps holds zero write permissions on any customer account, and that credentials are never stored. Cognito migration path documented under production roadmap.

Feedback 2 — Database Architecture: DynamoDB / NoSQL Evaluation

Recommendation: Consider DynamoDB or NoSQL to handle polymorphic AWS resource metadata.

How we have applied this feedback:

- This feedback pushed us to formally evaluate our database choice rather than treating it as a default. After studying the access patterns that AetherOps needs, we found that attack-path correlation — the

core feature of the system — requires joining resources to their relationships and traversing those connections graph-style, which is a relational workload.

- To address the polymorphic metadata concern Ms. Solanki raised, we have added a JSON attributes column to the resources table — giving us flexible storage per resource type (EC2 fields differ from S3 fields) while keeping the relational joins we need for the attack-path graph.
- We have added a formal data store evaluation section to our architecture document — crediting this feedback as the reason we conducted the analysis. DynamoDB is documented as the right path if the system scales to multi-account, multi-region enterprise volume.

Action taken: Architecture document updated with a data store decision rationale section. PostgreSQL + JSON column approach is explicitly justified against the NoSQL alternative. DynamoDB noted as the recommended path for enterprise-scale multi-account deployment.

Feedback 3 — Market Differentiation vs. Wiz, Orca, Prisma Cloud

Recommendation: Benchmark against CSPM / CNAPP platforms and define unique selling propositions beyond standard rule checking.

Competitive differentiation — what we have added:

Capability	Wiz / Orca / Prisma	AetherOps
Attack-path correlation with AI narration	☑ Yes (enterprise, $50K+/yr)	☑ Yes (free, open)
Plain-language AI explanation per finding	Partial	☑ Full — every finding and path
Cross-cloud cost comparison (AWS/Azure/GCP)	❌ Not a focus	☑ Live pricing APIs, AI-interpreted
Grounded AI chat bot on user's own data	❌ Not available	☑ Core feature
Target user: students, freelancers, small teams	❌ Enterprise-only pricing	☑ Explicitly designed for this
Self-hostable, zero licence cost	❌	☑

Action taken: Competitive comparison table added to project documentation. Our USP is now explicitly framed as: attack-path correlation + AI narration + cross-cloud cost comparison + grounded chat bot — all free and self-hostable for the segment that enterprise CSPM tools were never designed to serve.

Feedback 4 — Cost Modeling, Query Throttling & Architectural Scalability

Recommendation: Design a clear budget management, throttling, and scaling plan for multi-account environments.

What we are implementing in the current build:

- boto3 retry configuration with exponential backoff is being added to all AWS API calls. This directly handles AWS service throttling (TooManyRequestsException) without requiring any external queue or rate-limiter infrastructure.
- Gemini prompt size is bounded by design: the AI report sends the top 20 findings by severity, not all findings. This keeps the prompt within context limits and ensures the report focuses on what is actually critical rather than exhaustively listing every low-severity item.
- Scan jobs are run on-demand per user, not continuously — this naturally prevents runaway API costs on both the AWS and Gemini side.

Current scope and planned path:

- Multi-account scanning is outside the scope of this version. The current system is designed for a single AWS account per user. Multi-account support (AWS Organizations integration, cross-account aggregation) is documented as the primary production scaling path.

Action taken: boto3 retry config added to discovery module (in progress this week). Finding truncation logic added to the report generation prompt. A 'Known Scope Boundaries' section added to architecture documentation covering single-account scope, API throttling mitigation, and multi-account as the v2 roadmap item.

3. Active Work This Week

The team is currently in Week 2–3 of the implementation sprint. The following work is actively in progress:

- **Tushya & Dhruv:** Completing AWS resource discovery for EC2, S3, RDS, IAM, and Lambda — populating the resources and relationships tables.
- **Janki:** Building the Security Rules Engine — loading YAML rules, evaluating resources, writing findings to the database.
- **Priyanshi:** Building the React dashboard — resource inventory view, findings list, severity filters.
- **All:** Integrating mentor feedback into architecture documentation — decision rationale, competitive table, scope boundaries.

We are grateful to Ms. Mansi Solanki for the time and clarity she brought to Review 1. Her guidance has directly shaped what we are building this week — the boto3 retry logic, the formal database evaluation, the competitive positioning, and the production roadmap all stem from her feedback. We look forward to her continued mentorship and will share our next progress update at the end of this sprint.